

+ 4 times 16
+ 2 times 1

Relation Between Binary and Hex

As it turns out, computers are more comfortable with binary numbering system. In a binary system, there are only two digits 0 and 1. This means you can't count very far before you need to start using the next column:

0
1 last available digit
10 start using a new column
11
...
...

Binary numbering system is a natural system for computers because each of the thousands of electronic circuits in the computer can be in one of the two states—on or off. Thus, binary numbering system corresponds nicely with the circuits in the computer—0 means off, and 1 means on. 0 and 1 are called bits, a short-form of binary digits.

Hex numbers are used primarily as shorthand for binary numbers that the computers work with. Every hex digit represents four bits of binary information (Refer Figure D.1). In binary numbering system 4 bits taken at a time can give rise to sixteen different numbers, so the only way to represent each of these sixteen 4-bit binary numbers in a simple and short way is to use a base sixteen numbering system.

Suppose we want to represent a binary number 11000101 in a short way. One way is to find its decimal equivalent by multiplying each binary digit with an appropriate power of 2 as shown below:

$$1 * 2^7 + 1 * 2^6 + 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0$$

which is equal to 197.

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Figure D.1

Another method is much simpler. Just look at Figure D.1. From it find out the hex digits for the two four-bit sets (1100 and 0101). These happen to be C and 5. Therefore, the binary number's hex equivalent is C5. You would agree this is a easier way to represent the binary number than to find its decimal equivalent. In this method neither multiplication nor addition is needed. In fact, since there are only 16 hex digits, it's fairly easy to memorize the binary equivalent of each one. Quick now, what's binary 1100 in hex? That's right C. You are already getting the feel of it. With a little practice it is easy to translate even long numbers into hex. Thus, 1100 0101 0011 1010 binary is C53A hex.

As it happens with many unfamiliar subjects, learning hexadecimal requires a little practice. Try your hand at converting some binary numbers and vice versa. Soon you will be talking hexadecimal as if you had known it all your life.

E ***ASCII Chart***

There are 256 distinct characters used by IBM compatible family of microcomputers. Their values range from 0 to 255. These can be grouped as under:

Character Type	No. of Characters
Capital letters	26
Small-case Letters	26
Digits	10
Special Symbols	32
Control Character	34
Graphics Character	128
Total	256

Figure E.1

Out of the 256 character set, the first 128 are often called ASCII characters and the next 128 as Extended ASCII characters. Each ASCII character has a unique appearance. The following simple program can generate the ASCII chart:

```
main()  
{  
    int ch ;  
  
    for ( ch = 0 ; ch <= 255 ; ch++ )  
        printf ( "%d %c\n", ch, ch ) ;  
}
```

This chart is shown on the following page. Out of the 128 graphic characters (Extended ASCII characters), there are characters that are used for drawing single line and double line boxes in text mode. For convenience these characters are shown in Figure E.2.

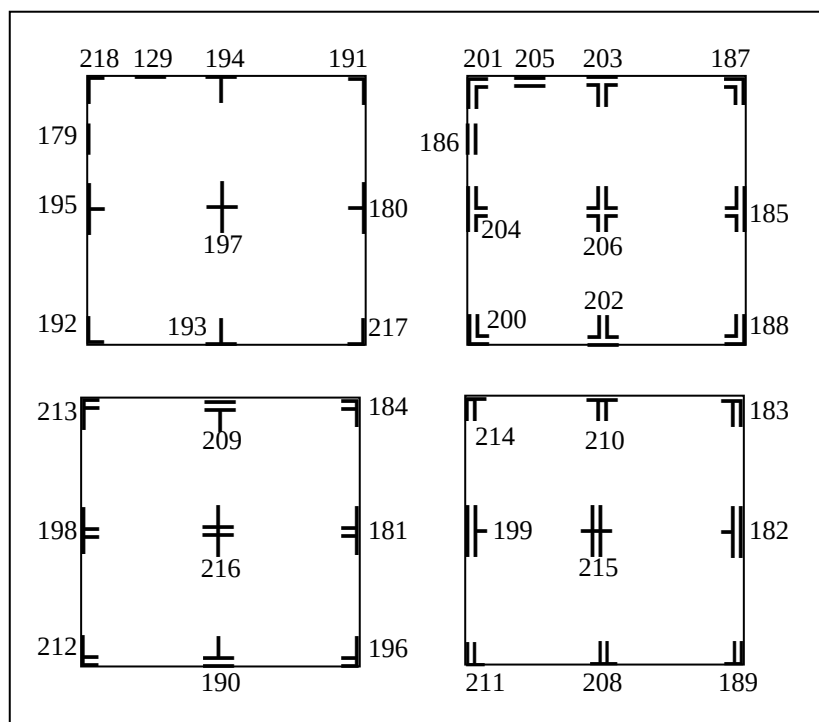


Figure E.2

Value	Char	Value	Char	Value	Char	Value	Char	Value	Char	Value	Char
0		22	—	44	,	66	B	88	X	110	n
1	☺	23	↕	45	-	67	C	89	Y	111	o
2	☹	24	↑	46	.	68	D	90	Z	112	p
3	♥	25	↓	47	/	69	E	91	[	113	q
4	♦	26	→	48	0	70	F	92	\	114	r
5	♣	27	←	49	1	71	G	93	]	115	s
6	♠	28	┐	50	2	72	H	94	^	116	t
7	●	29	↔	51	3	73	I	95	_	117	u
8	■	30	▲	52	4	74	J	96	`	118	v
9	○	31	▼	53	5	75	K	97	a	119	w
10	▣	32		54	6	76	L	98	b	120	x
11	☼	33	!	55	7	77	M	99	c	121	y
12	⊕	34	"	56	8	78	N	100	d	122	z
13	♪	35	#	57	9	79	O	101	e	123	{
14	🎵	36	\$	58	:	80	P	102	f	124	
15	☀	37	%	59	;	81	Q	103	g	125	}
16	▶	38	&	60	<	82	R	104	h	126	~
17	◀	39	'	61	=	83	S	105	i	127	^M H
18	↕	40	(	62	>	84	T	106	j	128	Ç
19	!!	41	)	63	?	85	U	107	k	129	ü
20	≡	42	*	64	@	86	V	108	l	130	é
21	§	43	+	65	A	87	W	109	m	131	â

Value	Char	Value	Char	Value	Char	Value	Char	Value	Char	Value	Char
132	ä	154	Ü	176	␣	198	␣	220	■	242	≥
133	à	155	é	177	␣	199	␣	221	■	243	<
134	å	156	£	178	␣	200	␣	222	■	244	┌
135	c	157	¥	179		201	␣	223	■	245	┐
136	ê	158	₣	180	└	202	␣	224	α	246	÷
137	ë	159	f	181	├	203	␣	225	β	247	≈
138	è	160	á	182	┤	204	␣	226	Γ	248	°
139	ï	161	í	183	␣	205	␣	227	π	249	•
140	î	162	ó	184	␣	206	␣	228	Σ	250	·
141	ì	163	ú	185	␣	207	␣	229	σ	251	√
142	Ä	164	ñ	186		208	␣	230	u	252	η
143	Å	165	Ñ	187	␣	209	␣	231	τ	253	²
144	É	166	ª	188	␣	210	␣	232	Φ	254	■
145	æ	167	º	189	␣	211	␣	233	θ	255	
146	Æ	168	¿	190	└	212	␣	234	Ω		
147	ô	169	┌	191	└	213	␣	235	δ		
148	ö	170	┐	192	└	214	␣	236	∞		
149	ò	171	½	193	└	215	␣	237	ø		
150	û	172	¼	194	└	216	␣	238	€		
151	ù	173	ı	195	└	217	└	239	∩		
152	ÿ	174	«	196	└	218	└	240	≡		
153	Ö	175	»	197	└	219	■	241	±		

***F* Helper.h File**

```

LRESULT CALLBACK WndProc ( HWND, UINT, WPARAM, LPARAM ) ;

HINSTANCE hInst ; // current instance

/* FUNCTION: InitInstance ( HANDLE, int
   PURPOSE: Saves instance handle and creates main window
   COMMENTS: In this function, we save the instance handle in a global
              variable and create and display the main program window.
*/
BOOL InitInstance ( HINSTANCE hInstance, int nCmdShow, char* pTitle )
{
    char classname[ ] = "MyWindowClass" ;
    HWND hWnd ;

    WNDCLASSEX wcex ;
    wcex.cbSize      = sizeof ( WNDCLASSEX ) ;
    wcex.style       = CS_HREDRAW | CS_VREDRAW ;
    wcex.lpfnWndProc = ( WNDPROC ) WndProc ;
    wcex.cbClsExtra  = 0 ;
    wcex.cbWndExtra  = 0 ;
    wcex.hInstance   = hInstance ;
    wcex.hIcon       = NULL ;
    wcex.hCursor     = LoadCursor ( NULL, IDC_ARROW ) ;
    wcex.hbrBackground = ( HBRUSH )( COLOR_WINDOW + 1 ) ;
    wcex.lpszMenuName = NULL ;
    wcex.lpszClassName = classname ;
    wcex.hIconSm     = NULL ;

    if ( !RegisterClassEx ( &wcex ) )
        return FALSE ;

    hInst = hInstance ; // Store instance handle in our global variable

    hWnd = CreateWindow ( classname, pTitle,
                        WS_OVERLAPPEDWINDOW,
                        CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, NULL,
                        NULL, hInstance, NULL ) ;

    if ( !hWnd )

```